Шум Перлина (Perlin Noise)

Шум Перлина (Perlin Noise)

Другая задача

Предыдущие алгоритмы - случайное блуждание, клеточные автоматы, BSP - работают с дискретными значениями. Ячейка - это либо стена, либо пол. Результат - подземелья, пещеры, комнаты.

Но что, если нам нужен ландшафт? Горы, равнины, реки, леса. Тут нет чёткой границы "стена или пол". Есть плавные переходы: равнина постепенно переходит в холмы, холмы - в горы. Вода у берега мелкая, дальше - глубокая.

Для этого нам нужны не дискретные значения (0 или 1), а **непрерывные** (от 0.0 до 1.0). Каждая ячейка матрицы хранит не тип тайла, а число. И это число должно плавно меняться от ячейки к ячейке, без резких скачков. Иначе мы снова получим шум.

Почему обычный рандом не подходит?

Допустим, мы генерируем "карту высот" - каждая ячейка хранит высоту рельефа. Если мы присвоим каждой ячейке случайное значение от 0.0 до 1.0, то рядом с горой (0.9) может оказаться впадина (0.1), а рядом с ней - снова гора (0.85). В реальном мире так не бывает. Ландшафт - плавный. Соседние точки имеют похожие значения.

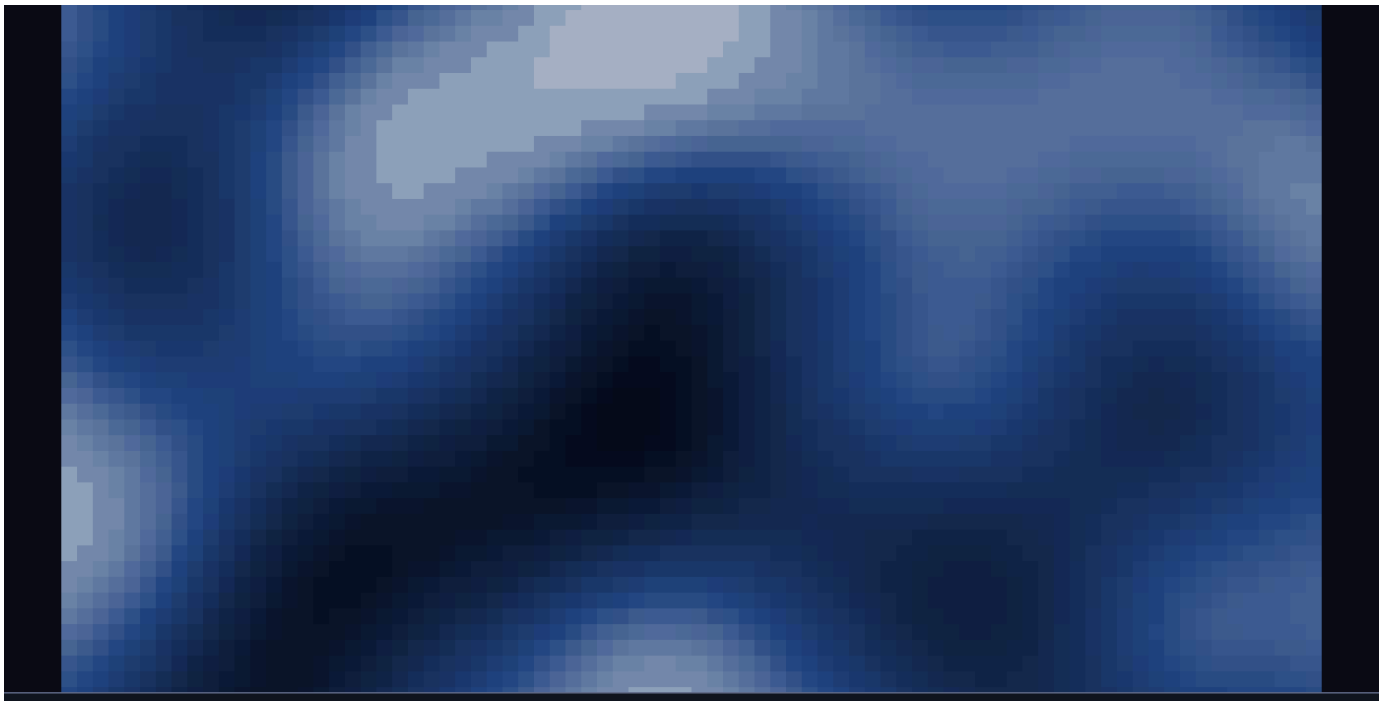
Нам нужен такой "рандом", где **соседние точки выдают близкие значения**, но при этом в целом картина выглядит случайной. Именно это делает шум Перлина.

Что такое шум Перлина?

Кен Перлин придумал этот алгоритм в 83-м для текстур в кино - и с тех пор он стал стандартом в геймдеве. По сути это "плавный рандом" - функция, которая для любых координат `(x, y)` возвращает значение, причём:

- Значения в соседних точках **близки** друг к другу.
- В целом картина выглядит **случайной** и естественной.
- Один и тот же вход всегда даёт **одинаковый** результат (детерминированность).

Грубо говоря, если обычный `random()` - это статический шум на телевизоре, то шум Перлина - это облака. Те же случайные паттерны, но плавные.



Шум Перлина — 1 октав(а)

Ключевые понятия

Прежде чем перейти к коду, разберём несколько терминов. Они пригодятся при настройке генератора.

Частота (Frequency)

Частота - это множитель входных координат. Когда мы вызываем `noise2d(x * frequency, y * frequency)` , мы как бы "растягиваем" или "сжимаем" сетку градиентов.

При `frequency=1` на карте 50×50 тайлов получится пара крупных "холмов". При `frequency=2` координаты удваиваются - мы проходим через вдвое больше ячеек сетки, и получается вдвое больше "холмов", но каждый вдвое мельче. При `frequency=4` - ещё мельче. Грубо говоря, частота определяет, сколько "бугров" помещается на карте.

Амплитуда (Amplitude)

Амплитуда - множитель выходного значения. `noise2d` возвращает число примерно от -1 до 1. Если умножить на `amplitude=0.5` , диапазон сжимается до [-0.5, 0.5]. Т.е. амплитуда определяет, насколько сильно этот слой шума влияет на итоговую карту.

¶

Октавы (Octaves)

Реальный ландшафт - это не одна волна, а наложение волн разного масштаба. Горный хребет (крупный масштаб) + отдельные вершины (средний) + камни и трещины (мелкий).

Октавы - это именно это. Мы вызываем `noise2d` несколько раз, каждый раз **удваивая** частоту и **уменьшая** амплитуду вдвое. Почему удваиваем? Термин "октава" взят из музыки: октава - это удвоение частоты звука. Тот же принцип: каждый следующий слой шума вдвое мельче предыдущего. Это даёт **самоподобие** - ландшафт выглядит похоже на любом масштабе, как фрактал.

Как это работает?

Ок, звучит как магия: "плавный рандом". Но на самом деле алгоритм довольно понятный. Четыре шага:

1. **Сетка градиентов.** Представьте координатную сетку. В каждом узле (целочисленные координаты) стоит случайный вектор - **градиент**. Он указывает в какую-то сторону.
2. **Скалярное произведение.** Для произвольной точки `(x, y)` находим четыре ближайших узла. От каждого узла проводим вектор к нашей точке и берём **скалярное произведение** (dot product) с градиентом этого узла. Получаем четыре числа.
3. **Сглаживание.** Нам нужно смешать эти четыре числа. Но если линейно интерполировать - на границах ячеек будут видны резкие переходы. Поэтому перед интерполяцией мы пропускаем дробные координаты через **fade-функцию** - кривую, которая плавно переходит от 0 к 1.
4. **Интерполяция.** Билинейная интерполяция четырёх значений с весами из fade-функции. Результат - одно число, плавно меняющееся при движении по сетке.

Т.е. каждый узел сетки "тянет" значение в свою сторону (через градиент), а fade + интерполяция обеспечивают плавный переход. Получается - одинаковые координаты всегда дают одинаковый результат, но при этом картина выглядит случайной.

Таблица перестановок и градиенты

Нам нужен способ для каждого узла сетки получить "случайный" градиент. Но этот рандом должен быть **детерминированным** - одни и те же координаты всегда дают один и тот же градиент. Для этого используют **таблицу перестановок** - перемешанный массив чисел от 0 до 255, который удваивается, чтобы не проверять выход за границы. Координаты хешируются через эту таблицу: `perm[perm[x] + y]` - и результат определяет градиент.

Градиенты - 8 направлений: по осям и по диагоналям.

Python C#

```python
import math
import random

def make_perm(seed=0):
    rng = random.Random(seed)
    p = list(range(256))
    rng.shuffle(p)
    return p + p  # double to avoid overflow

PERM = make_perm(0)

# 8 gradient directions


```

```
GRADS = [(1,0), (-1,0), (0,1), (0,-1),
         (1,1), (-1,1), (1,-1), (-1,-1)]
```

Seed таблицы перестановок определяет "мир". Один seed - одна карта, всегда одинаковая. Другой seed - другая карта. Подробнее о seed-ах - в Случайность и вероятность.

Fade - сглаживающая кривая

Если интерполировать линейно, на границах ячеек будут видны швы - карта получится "решётчатая". Fade-функция $6t^5 - 15t^4 + 10t^3$ решает эту проблему: её первая и вторая производные равны нулю на краях (0 и 1), поэтому переходы между ячейками абсолютно гладкие.

Python C#

```python
def fade(t):
    # 6t^5 - 15t^4 + 10t^3
    return t * t * t * (t * (t * 6 - 15) + 10)
```

Lerp и Grad

`lerp` - линейная интерполяция: `a + t * (b - a)`. `grad` - выбирает градиент по хешу и считает скалярное произведение с вектором от узла до точки. Всё скалярное произведение двух 2D-векторов - это `g.x * dx + g.y * dy`.

Python C#

```python
def lerp(a, b, t):
    return a + t * (b - a)

def grad(hash_val, x, y):
    g = GRADS[hash_val % 8]
    return g[0] * x + g[1] * y
```

Noise2D - шум для одной точки

Ключевая функция. Для точки `(x, y)`:

1. Берём целую часть координат через `floor` и маскируем `& 255` - получаем индексы `xi`, `yi` в таблице перестановок.
2. Дробная часть `xf = x - floor(x)`, `yf = y - floor(y)` - позиция внутри ячейки (от 0.0 до 1.0).
3. Пропускаем `xf` и `yf` через `fade` - получаем сглаженные веса `u` и `v`.
4. Хешируем четыре угла ячейки через таблицу перестановок: `aa = perm[perm[xi] + yi]`, `ab = perm[perm[xi] + yi+1]`, `ba = perm[perm[xi+1] + yi]`, `bb = perm[perm[xi+1] + yi+1]`.
5. Для каждого угла вызываем `grad(hash, dx, dy)`, где `dx`, `dy` - расстояние от угла до точки. Для левого нижнего: `(xf, yf)`. Для правого нижнего: `(xf-1, yf)`. И так далее.
6. Двойная интерполяция: сначала `lerp` по X для нижней и верхней пары, потом `lerp` по Y между ними.

Результат - число примерно от -1 до 1, плавно меняющееся при движении по координатам.

Октавы (fBm)

Одна октава шума - плавные, крупные формы. Ну, как горный хребет, нарисованный одной волной. Ни камней, ни трещин, ни мелких неровностей. А реальный ландшафт - это наложение деталей разного масштаба: хребет + отдельные вершины + камни + трещины.

fBm (fractional Brownian motion) решает это: складываем несколько октав шума, каждая - с удвоенной частотой и уменьшенной амплитудой.

Допустим, у нас 4 октавы:

Октава	Частота	Амплитуда	Что добавляет
1	1×	1.0	Общая форма - континенты, океаны
2	2×	0.5	Горные хребты, крупные заливы
3	4×	0.25	Отдельные холмы, бухты
4	8×	0.125	Камни, мелкие неровности

Каждая следующая октава - вдвое мельче и вдвое слабее. Складываем всё и нормализуем.

Python C#

```python
value = 0.0
amplitude = 1.0
frequency = 1.0
```

```
max_amp = 0.0

for i in range(octaves):
    value += noise2d(x / scale * frequency,
                     y / scale * frequency) * amplitude
    max_amp += amplitude
    amplitude *= 0.5   # persistence
    frequency *= 2.0   # lacunarity

# normalize to [0, 1]
result = (value / max_amp + 1) / 2
```

Влияние параметров

- **Persistence** (0.5 в нашем примере) - множитель амплитуды между октавами. При 0.5 каждая следующая октава вносит вдвое меньше. При 0.3 - детали почти незаметны, ландшафт очень гладкий. При 0.7 - мелкие детали доминируют, получается "шершавый" рельеф.
- **Lacunarity** (2.0) - множитель частоты. Обычно 2.0, но можно экспериментировать: при 3.0 каждая октава втрое мельче, детали появляются резче.
- **Octaves** - количество слоёв. 1 октава - плавные холмы без деталей. 4 - хороший баланс. 8 - микрорельеф. Больше 6 обычно не нужно: вклад каждой следующей октавы экспоненциально падает.
- **Scale** - базовый масштаб первой октавы. При `scale=10` получатся мелкие острова, при `scale=50` - огромные континенты.

Попробуйте менять persistence: при 0.3 карта выглядит как пустыня - почти плоская, только крупные формы. При 0.7 - как скалистые горы, всё покрыто мелкими неровностями. Persistence 0.5 - разумный дефолт.

От шума к тайлам

Ок. У нас есть матрица чисел от 0.0 до 1.0. Как превратить её в игровой уровень? С помощью **пороговых значений** (threshold). Для каждой ячейки сравниваем значение с порогами и назначаем тип тайла:

Python C#

```
# for each cell with noise value:
if value < 0.3:
    tile = "water"
elif value < 0.5:
    tile = "sand"
elif value < 0.7:

```

```
    tile = "grass"
elif value < 0.85:
    tile = "forest"
else:
    tile = "mountain"
```

Меняя пороги, мы управляем тем, сколько воды, гор и т.д. будет на карте. Низкий порог для воды (0.2) - мало воды, высокий (0.4) - архипелаг.

Пещеры через шум

Хм, а что если нам нужен не ландшафт, а пещеры? Тот же принцип, только порог один: значение выше 0.5 - стена, ниже - пол. Получается карта с плавными, органическими пещерами - похоже на результат клеточных автоматов, но с большим контролем.

Трюк в параметрах. Низкая частота (большой `scale`) даёт крупные открытые залы. Высокая (маленький `scale`) - узкие извилистые тоннели. А если сместить порог с 0.5 на 0.4 - пещеры станут просторнее, на 0.6 - теснее.

Ещё один приём: **комбинация двух шумов**. Один с крупным масштабом определяет общую форму пещеры, второй с мелким - добавляет неровности стен. Перемножаем значения - и получаем пещеры с гладкими крупными формами и шероховатыми деталями.

Terraria и Minecraft используют именно этот подход: шум Перлина с разными масштабами и порогами для генерации пещер, руды, биомов - всего одним алгоритмом.

Связность пещер, как и у клеточных автоматов, не гарантируется. Может потребоваться валидация или прорезание коридоров между изолированными регионами.

Perlin Worms - червоточины

А что, если нужны не залы, а именно тоннели? Длинные, извилистые, как червоточины? Для этого есть приём **Perlin Worms**: "червяк" ползёт по карте, и его направление на каждом шаге определяется шумом Перлина.

Идея: берём точку `(x, y)` - голову червяка. Вызываем `noise2d(x, y)` - получаем значение от -1 до 1. Умножаем на `2π` - получаем угол. Двигаем голову на один шаг в этом направлении. Вокруг головы прорезаем круг радиусом 2-3 тайла. Повторяем сотни раз.

Результат - плавные, естественно выглядящие тоннели. Червяк не дёргается (потому что шум Перлина плавный), не ходит по прямой (потому что шум всё-таки случайный). Получается нечто среднее между случайным блужданием (которое слишком хаотично) и ручным рисованием (которое слишком ровно).

Можно запустить несколько червяков из разных точек - получится система тоннелей. А если комбинировать с пороговыми пещерами выше - тоннели соединяют залы.

Наложение слоёв

Одна карта шума даёт нам высоту. Но ландшафт - это не только высота. Есть ещё влажность, температура, плотность растительности. И каждый из этих параметров можно генерировать отдельной картой шума.

Например, комбинация двух карт:

- **Карта высот**: определяет рельеф (вода, равнина, горы).
- **Карта влажности**: определяет, сухой регион или влажный.

Из их комбинации получаются **биомы**:

- Низко + влажно → болото.
- Низко + сухо → пустыня.
- Высоко + влажно → густой лес.
- Высоко + сухо → каменистые горы.

Создаёте **две** карты шума с разными `seed` и разным масштабом (например, `scale * 1.5` для влажности - чтобы зоны не совпадали с рельефом). Затем для каждой ячейки берёте два значения - `h` (высота) и `m` (влажность) - и через цепочку `if/elif` определяете биом. Приоритет: сначала проверяем экстремальные значения высоты (вода, горы), затем влажность (лес, пустыня), остальное - трава.

Обратите внимание, что для двух карт мы используем **разные seed** (1 и 2) и **разный масштаб**. Если использовать одинаковые - карты совпадут, и смысл в наложении пропадёт.

Количество слоёв ничем не ограничено. Можно добавить карту температуры, карту ветров, карту магической энергии - всё, что требует ваш игровой мир.

Бесконечная генерация

Важное свойство шума Перлина: он не привязан к размеру матрицы. Функция шума принимает любые координаты и возвращает значение. Т.е. нам не нужно генерировать всю карту заранее - мы можем генерировать только ту часть, которую видит игрок.

Игрок сдвинулся вправо? Генерируем новый столбец тайлов. Сдвинулся вверх? Новую строку. При этом уже сгенерированные тайлы не меняются - функция детерминированная.

Это позволяет создавать "бесконечные" миры, как в Minecraft или Terraria, где карта генерируется по мере исследования.

Плюсы и минусы

- **Естественность**: результат выглядит как настоящий ландшафт. Плавные переходы, никакой "решётчатости".
- **Бесконечность**: можно генерировать любую область по координатам, не храня всю карту в памяти.

- **Воспроизводимость**: один seed - одна карта. Всегда.
- **Слои**: легко комбинировать несколько карт для создания сложных биомов.

Но:

- **Нет структуры**: шум Перлина не создаёт комнаты с коридорами. Пещеры - да, но если нужны чёткие прямоугольные комнаты и двери - используйте BSP.
- **Настройка параметров**: scale, octaves, пороги биомов - всё это нужно подбирать экспериментально. Универсальных значений нет.

Что дальше?

Шум Перлина хорошо работает для открытых миров и ландшафтов. Но если ваша игра - подземелье или рогалик, вам скорее пригодятся:

- BSP-генерация - комнаты и коридоры.
- Клеточные автоматы - естественные пещеры.
- Случайное блуждание - простейшая генерация проходов.

А ещё можно комбинировать: BSP для структуры подземелья, а шум Перлина - для генерации поверхности над ним.

Если хотите разобраться в seed-ах, детерминированности и управлении случайностью - см. Случайность и вероятность.

А если интересны другие подходы к процедурной генерации:

- Wave Function Collapse - генерация из правил соседства.
- L-системы - формальные грамматики для растений и фракталов.

Powered by Wiki.js